

HelixTrack

HelixTrack

JIRA — альтернатива для свободного мира.

Краткое описание

HelixTrack — современная комплексная open-source-альтернатива JIRA (а также Confluence благодаря расширению Documents), представляющая собой кроссплатформенную систему управления проектами и отслеживания задач. Она построена на базе микросервисной архитектуры Go и включает нативные клиенты для веб, десктопа и мобильных платформ.

Краткая аннотация

Open-source-альтернатива JIRA/Confluence. Микросервисная архитектура Go («HelixTrack Core») предоставляет унифицированный REST API для управления проектами и задачами, а также рабочее пространство документов в стиле Confluence. Доступ осуществляется через нативные клиенты для Web, Desktop, Android и iOS по протоколу HTTP/3 QUIC.

Подробное описание

HelixTrack — это open-source-платформа для управления проектами и отслеживания задач, позиционируемая как альтернатива для свободного мира вместо JIRA и Confluence. Это полноценная замена двум продуктам, к которым привязано большинство инженерных команд, но переработанная в виде ПО, которое вы полностью контролируете и можете развернуть где угодно. В основе системы лежит **HelixTrack Core** — микросервис на базе REST API, написанный на Go с использованием фреймворка Gin. Он предоставляет полноценное отслеживание задач, agile- и scrum-доски, управление командами и иерархическую систему

прав доступа, реализация которой может быть заменена с локальной на HTTP-сервис. Это позволяет масштабировать модель авторизации от одного ноутбука до распределённого кластера без изменений в коде приложения.

Вместо того чтобы разрастаться до десятков маршрутов, Core концентрирует всё взаимодействие через единую конечную точку /do с маршрутизацией по действиям и унифицированным форматом запросов и ответов (action/jwt/object/data на входе, errorCode/errorMessage/data на выходе). Все клиенты работают по единому компактному контракту, а добавление новой функциональности сводится к добавлению действия, а не нового URL, который нужно документировать, защищать и поддерживать.

Core интегрируется с разделёнными сервисами аутентификации, управления правами и локализации, которые общаются по протоколу HTTP/3 QUIC и могут работать на отдельных машинах или кластерах, а также полностью отключаться в тестовых конфигурациях. Данные хранятся в SQLite для быстрого развёртывания в разработке и в PostgreSQL в продакшене, причём по умолчанию шифруются на диске с помощью SQLCipher (AES-256). Это гарантирует защиту конфиденциальных данных проекта не как опцию, а как стандарт.

Расширение **Documents V2** превращает трекер в полноценную платформу знаний: рабочее пространство в стиле Confluence с пространствами, страницами, контролем версий, шаблонами, совместной работой в реальном времени (WebSocket) и аналитикой. Теперь вики и трекер задач существуют в рамках одного бэкенда, а не как два отдельных продукта, сшитых вместе.

Вокруг Core реализованы клиентские приложения: Angular-клиент для веба, десктопное приложение на базе Tauri + Angular, нативные приложения для Android (Kotlin) и iOS (Swift), а также клиенты для HarmonyOS и Aurora OS, плюс скринсейвер. Все они взаимодействуют с одним бэкендом и автоматически обнаруживают его в локальной сети через UDP-широковещание, поэтому новый клиент находит сервер без ручной настройки. Клиентские приложения поддерживаются в отдельных закрытых репозиториях и представлены здесь только на уровне продукта.

Почему мы это создали

Чтобы предложить командам по-настоящему открытую и самостоятельно развёртываемую альтернативу стеку JIRA + Confluence — «для свободного мира» — без привязки к

вендору, объединив корпоративные инструменты отслеживания задач, документооборота и совместной работы под одной лицензией с открытым исходным кодом.

Почему это меняет правила игры

Платформа объединяет два тяжёлых коммерческих продукта — систему отслеживания задач и вики-движок с документами — в единую открытую, высокопроизводительную и самостоятельно развёртываемую систему, дополняя их тем, чего никогда не предлагали конкуренты: настоящими кроссплатформенными *нативными* клиентами (веб, десктоп, Android, iOS, а также HarmonyOS и Aurora OS), работающими на базе единого бэкенд-контракта. Ключевое преимущество — полный контроль без компромиссов. Архитектура HTTP/3, повсеместно реализованная в виде полностью разделённых микросервисов, и сквозное шифрование данных в состоянии покоя по стандарту SQLCipher AES-256 обеспечивают производительность и уровень безопасности, обычно доступные только в проприетарных SaaS-решениях, но в системе, которую вы развёртываете самостоятельно. Никаких лицензий на рабочие места, привязки к вендору или утечки данных за пределы вашей инфраструктуры. Команды получают привычный опыт работы с JIRA и Confluence на собственном оборудовании, под одной лицензией с открытым исходным кодом.

Что здесь инновационного

- Единая система действий на базе /do API — один эндпоинт, одна обёртка, маршрутизация по действиям. Новые возможности появляются как новые действия, а не новые URL, что сокращает поверхность атаки, объём клиентского кода и нагрузку на документацию до одного общего для всех платформ контракта.
- HTTP/3 QUIC как *стандартный* транспорт между сервисами — современная низколатентная сеть с устойчивостью к разрывам соединений между сервисами с первого дня, а не дописанная постфактум.
- Движок прав доступа, который можно подменять между локальной внутрипроцессной реализацией и HTTP-сервисом, а также дополнительные независимо развёртываемые сервисы аутентификации, управления правами и локализации — единая модель авторизации вне зависимости от того, работает ли один процесс или кластер.
- Изоляция данных между пространствами с помощью флага --space-root: каждый проект получает собственную изолированную базу данных и хранилище активов,

благодаря чему арендаторы и проекты разделяются на уровне хранения, а не фильтрами запросов.

- Шифрование данных в состоянии покоя по стандарту SQLCipher AES-256 — конфиденциальные данные проектов защищены на диске по умолчанию и прозрачно для пользователя.
- Автоматическое обнаружение клиентом сервера через UDP-широковещание в локальной сети — клиент находит Core без ручной настройки.
- Документы V2 — полноценная «альтернатива Confluence» с параллельным редактированием на основе оптимистичной блокировки, обнаружением конфликтов и полной историей изменений — настоящие совместные документы, работающие на том же бэкенде, что и трекер задач.

Главные технические вызовы и как мы их преодолели

- **Шесть клиентских платформ, один бэкенд, ноль дрейфа контрактов.** Поддержка клиентов для Web/Angular, Desktop/Tauri, Android/Kotlin, iOS/Swift, HarmonyOS и Aurora обычно означает шесть расходящихся интеграций API, которые со временем теряют синхронизацию. Мы устранили этот риск, сделав единый маршрутизируемый по действиям эндпоинт /do API и его фиксированную обёртку *единственным* контрактом — все клиенты работают с ним идентично — и добавив поверх UDP-широковещание для обнаружения сервисов, чтобы клиенты находили Core в сети без ручной настройки эндпоинтов.
- **Разделение сервисов без потери производительности.** Выделение аутентификации, управления правами и локализации в независимо развёртываемые сервисы обычно добавляет сетевой хоп на каждый вызов. Мы внедрили HTTP/3 QUIC для всех межсервисных вызовов, чтобы эти переходы оставались быстрыми и устойчивыми к разрывам соединений, а также сделали каждый сервис независимо запускаемым — даже полностью отключаемым в тестовых конфигурациях — так что разделение становится вопросом выбора при развёртывании, а не фиксированной нагрузкой.
- **Совместная работа уровня Confluence без потери данных.** Многопользовательское редактирование в реальном времени чревато конфликтующими изменениями. Документы V2 решают эту проблему с помощью пространств/страниц/версионирования на основе оптимистичной блокировки, явного обнаружения конфликтов, полной истории изменений и синхронизации в реальном времени через WebSocket — совместная работа остаётся согласованной, а не молча перезаписывает правки.

- **Шифрование данных в состоянии покоя без потери производительности.** SQLCipher AES-256 защищает данные на диске, но добавляет накладные расходы на каждый запрос. Мы компенсировали это многоуровневым кэшированием (LRU в памяти перед Redis в сервисе локализации), так что горячие пути, например многоязычные запросы, остаются быстрыми, а данные — зашифрованными.

Технологический стек

- **Go + Gin** — выбраны для высокопроизводительных HTTP-сервисов с низкой задержкой и возможностью деплоя в виде единого бинарного файла; включают ядро REST API, его JWT/CORS-посредник и маршрутизатор /do, обслуживающий всю систему.
- **HTTP/3 QUIC** — выбран в качестве транспортного протокола между ядром и сервисами аутентификации, разграничения прав доступа и локализации, поскольку мультиплексная архитектура QUIC с миграцией соединений сокращает задержки в хвосте распределения и устойчива к нестабильным каналам связи, где TCP зависает.
- **PostgreSQL (прод) / SQLite (дев)** — единая реляционная модель, поддерживающая сложную схему трекинга и документооборота на обеих платформах: SQLite обеспечивает разработку без предварительной настройки и хранение данных в файлах, а Postgres берёт на себя нагрузку в продакшене через выделенный профиль production в Docker Compose.
- **SQLCipher (AES-256)** — выбран для прозрачного шифрования данных на уровне базы, благодаря чему защита конфиденциальной информации не требует дополнительной криптографии на уровне приложения и не влияет на написание запросов.
- **Redis** — используется как общий кэш-уровень за in-memory LRU-кэшем в сервисе локализации, образуя двухуровневую систему кэширования, которая обеспечивает быстрый доступ к часто запрашиваемым многоязычным данным даже при наличии накладных расходов на шифрование.
- **Uber Zap + Lumberjack** — выбраны для структурированного, низкоаллокационного логирования с автоматическим ротацией логов, что позволяет поддерживать наблюдаемость системы в продакшене без неконтролируемого роста объёма логов.
- **golang-jwt / JWT** — используются как механизм бесстатусной аутентификации; подписанный токен передаётся в поле jwt каждого запроса к /do, обеспечивая единообразие аутентификации для всех клиентов.

- **Angular 19 (+ Material, RxJS)** — выбран для создания реактивного, компонентно-ориентированного веб-клиента с готовой системой дизайна Material.
- **Tauri 2.0 + Rust + Angular** — выбраны для поставки нативной десктопной оболочки с минимальным размером за счёт повторного использования UI Angular внутри вебвью на базе Rust, что позволяет избежать упаковки полноценного браузерного движка.
- **Kotlin (Android) / Swift + SwiftUI (iOS)** — выбраны для предоставления мобильным пользователям по-настоящему нативных клиентов, соответствующих платформенным стандартам, вместо обёрнутого веб-представления.
- **Docker / Docker Compose (совместимо с Podman)** — выбраны для воспроизводимого контейнеризованного деплоя с предустановленными проверками /health, а также совместимости с Podman, что исключает зависимость от конкретного демона или вендора.
- **Testify (Go); Cypress/Playwright/Karma+Jasmine (клиенты)** — используются для многоуровневого автоматизированного тестирования, покрывающего контракт бэкенда и пользовательские интерфейсы клиентов независимо друг от друга, что соответствует архитектуре «один бэкенд — множество клиентов».

Статус и замечания по прозрачности

- **Статус: бета.** Ядро HelixTrack представляет собой работающий микросервис REST API; расширение **Documents V2** документировано как завершённое примерно на 95%, однако в нём присутствует известная проблема с маппингом полей в базе данных, поэтому оно не позиционируется как полностью готовое к релизу.
- **Лицензия: не определена.** В файле CLAUDE.md указана лицензия MIT, однако в core/LICENSE прописана Apache 2.0 — это расхождение должно быть устранено до окончательного утверждения лицензии.
- Приведённые в README проекта показатели производительности (например, 50 000+ запросов в секунду, время выполнения запросов менее миллисекунды) заявлены как целевые/маркетинговые ориентиры, а не как результаты независимых бенчмарков, поэтому они не включены в приведённые выше утверждения.
- Клиентские приложения (веб, десктоп, Android, iOS, Aurora, HarmonyOS) находятся в **закрытых** репозиториях и описаны только на уровне продукта.

Приоритетный уровень: Helix-primary и флагман линейки продуктов Helix-Track — имеют приоритет над любыми проектами Server Factory.